

GNN-MPC Cost Function for Square-Rod Forging

Agility Forge — control progress notes

2026-09-26

1 Receding-horizon setup

At each real hit t , the MPC measures the bar's surface displacement $\mathbf{x}_t \in \mathbb{R}^{N \times 3}$: one displacement vector $\mathbf{x}_{t,i} = (x_{t,i}, y_{t,i}, z_{t,i})$ for each of the $N \approx 4400$ surface nodes. The component x runs along the bar (clamp to free end); y and z lie across it, in the plane of a cross-section.

It then plans the next $K = 10$ hits. Hit k has three controls,

$$\mathbf{u}_k = (d_k, R_k, s_k), \quad k = 1, \dots, K,$$

where d_k is the station (where the 19.3 mm die band starts along the bar), R_k is the angle the bar is turned before the hit, and s_k is the stroke (how far each die moves in, in mm).

The GNN (message-passing network f_{GNN}) predicts the shape after each planned hit, starting from the measured state:

$$\hat{\mathbf{x}}_0 = \mathbf{x}_t, \quad \hat{\mathbf{x}}_k = \hat{\mathbf{x}}_{k-1} + f_{\text{GNN}}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k), \quad k = 1, \dots, K.$$

Only the first planned hit $\mathbf{u}_1$ is applied to the simulator. The simulator returns the real new shape $\mathbf{x}_{t+1}$, and the MPC plans again.

2 Optimization problem

$$\min_{\mathbf{u}_1, \dots, \mathbf{u}_K} J \quad \text{subject to} \quad \begin{cases} d_{\min} \leq d_k \leq d_{\max} \\ 0^\circ \leq R_k \leq 360^\circ \\ 0 \leq s_k \leq s_{\max} \end{cases} \quad k = 1, \dots, K, \quad (1)$$

with $d_{\min} = 17.7$ mm (the 800 °C point of the initial temperature profile), $d_{\max} = 75.3$ mm, and $s_{\max} = 2$ mm.

3 Cost function

$$J = E + w E_\perp + \frac{E_0}{K} \left[e P_s + P_\Delta \right] \quad (2)$$

where

- (1) E is the **error term**: distance of the predicted shape from the target;
- (2) $E_{\perp}$ is the **cross-section term**, weighted by w ;
- (3) P_s is the **stroke-size penalty**, weighted by e ;
- (4) P_{Δ} is the **control-change penalty**, with its own weights $\lambda_d, \lambda_R, \lambda_s$.

Each term is defined below.

(1) Error term (always used). Each surface node’s squared distance from its position in the target shape $\mathbf{x}^*$, summed over all nodes and all planned hits:

$$E = \sum_{k=1}^K \sum_{i=1}^N \|\hat{\mathbf{x}}_{k,i} - \mathbf{x}_i^*\|^2 = \sum_{k=1}^K \sum_{i=1}^N \left[(\hat{x}_{k,i} - x_i^*)^2 + (\hat{y}_{k,i} - y_i^*)^2 + (\hat{z}_{k,i} - z_i^*)^2 \right]. \quad (3)$$

For the square target, about 99% of E at the start comes from the first (lengthwise) part, because the target bar is 35 mm longer than the billet.

(2) Cross-section term. The same distance, counting only the across-the-bar directions y and z :

$$E_{\perp} = \sum_{k=1}^K \sum_{i=1}^N \left[(\hat{y}_{k,i} - y_i^*)^2 + (\hat{z}_{k,i} - z_i^*)^2 \right]. \quad (4)$$

With weight w , a sideways miss costs $(1 + w)$ times as much as the same miss along the bar. This makes the cross-section shape count in the cost.

(3) Stroke-size penalty. With the stroke scaled to $[0, 1]$, $\tilde{s}_k = s_k / s_{\max}$:

$$P_s = \sum_{k=1}^K \tilde{s}_k^2. \quad (5)$$

(4) Control-change penalty. With the station scaled to $[0, 1]$, $\tilde{d}_k = (d_k - d_{\min}) / (d_{\max} - d_{\min})$, and $\mathbf{u}_0$ the last hit actually applied:

$$P_{\Delta} = \sum_{k=1}^K \left[\lambda_d (\tilde{d}_k - \tilde{d}_{k-1})^2 + \lambda_R \sin^2(R_k - R_{k-1}) + \lambda_s (\tilde{s}_k - \tilde{s}_{k-1})^2 \right]. \quad (6)$$

The angle uses $\sin^2$ because the two dies press opposite sides of the bar, so turning by 0° or 180° gives the same hit; the penalty is largest for a 90° change.

Scaling of (3) and (4). E_0 is the value of the error term E at the optimizer’s starting guess, so E_0/K is one hit’s share of the plan’s starting error. A penalty weight of 1 therefore means “a unit penalty on one hit costs as much as one hit’s worth of error”, independent of the error’s size ($\sim 10^6 \text{ mm}^2$ at the start). The cross-section weight w is not scaled this way; it is in the error’s own units.

4 Weights used in each experiment

M is the number of message-passing steps in the GNN that did the planning.

Experiment	M	w	e	λ_d	λ_R, λ_s
E0: first MPC run (original cost)	5	0	0	0	0
E1: change penalty, all controls	3	0	0	0.01, 0.1, 1	0.01, 0.1, 1
E1: change penalty, station only	3	0	0	0.01, 0.1, 1	0
E2: cross-section / stroke size	3	30, 100, 300	0, 0.1, 1	0	0
E3: refinement grid	3	30, 50, 100	0.03, 0.1, 0.3	0	0
E4: robustness (5 seeds each)	3	30 or 100	0.1 or 0.3	0	0
Real-simulator runs	3	30 or 100	0, 0.1 or 0.3	0	0
Lead design	3	30	0.3	0	0

With the lead design the cost reduces to

$$J_{\text{lead}} = E + 30 E_{\perp} + 0.3 \frac{E_0}{K} \sum_{k=1}^K \left(\frac{s_k}{2 \text{ mm}} \right)^2. \quad (7)$$

5 Solver

J is minimized with SLSQP (sequential quadratic programming with a quasi-Newton Hessian), using exact gradients $\partial J / \partial \mathbf{u}$ obtained by back-propagating through the K chained GNN predictions. For numerical conditioning the optimizer works on controls rescaled to $[0, 1]$ and on J divided by its value at the starting guess; this changes the units only, not the minimizer. The starting guess is the previous plan shifted by one hit (its last hit repeated).